

Progetto finale: Integrazione di un modello di Machine Learning in una web app Flask (MVT)

Contesto

Nel progetto precedente hai addestrato un modello di Machine Learning per la valutazione di una richiesta di mutuo.

Il modello:

- è stato costruito in Python,
- è stato addestrato su un dataset tabellare,
- è stato esportato in formato `.joblib`,
- include una pipeline completa di preprocessing e classificazione.

In questo esercizio devi portare la stessa logica in un'applicazione web, utilizzando **Flask**, seguendo un'architettura ispirata al modello **MVT (Model · View · Template)**.

Obiettivo

Realizzare una piccola applicazione Flask che permetta di:

1. visualizzare una pagina iniziale con un form di inserimento dati,
 2. acquisire i dati inseriti dall'utente,
 3. costruire una struttura dati compatibile con il modello esportato,
 4. caricare automaticamente il file `.joblib`,
 5. ottenere una previsione tramite il modello,
 6. reindirizzare l'utente verso una pagina di esito positivo o negativo.
-

◆ Stack richiesto

- Python
- Flask
- joblib
- pandas

- HTML (template Jinja2)

- ⚠ Non è richiesta autenticazione
 - ⚠ Non è richiesto CSS avanzato
 - ⚠ Non è richiesto database
 - ⚠ Non è richiesta validazione lato client avanzata
-

♦ Architettura MVT (semplificata)

Model → logica di caricamento e utilizzo del modello `.joblib`

View → route Flask, acquisizione dati del form, chiamata al modello e gestione del routing

Template → pagine HTML da mostrare all'utente

Separare chiaramente:

- logica di caricamento del modello
- logica di previsione
- gestione delle richieste HTTP
- rendering HTML

Questa separazione riprende esplicitamente l'impostazione MVT già richiesta nel progetto Creature Loader.

Modello logico (Model)

L'applicazione deve contenere una parte dedicata alla gestione del modello di Machine Learning.

Questa parte deve:

- caricare il file `modello_mutuo.joblib`,
- ricevere in input i dati del form,
- costruire un `DataFrame` con le colonne attese dal modello,
- restituire la previsione finale.

- ⚠ Il file `.joblib` deve essere trattato come risorsa esterna già pronta
- ⚠ Non è richiesto riaddestrare il modello
- ⚠ Non è richiesto modificare il notebook di training

Dati di input

L'utente dovrà compilare un form HTML contenente i campi richiesti dal modello.

I nomi dei campi del form dovranno essere coerenti con le feature attese dal modello esportato.

Esempi possibili:

- genere
- stato civile
- numero di figli
- istruzione
- lavoro autonomo
- reddito richiedente
- reddito co-richiedente
- importo mutuo
- durata mutuo
- storico creditizio
- area geografica

⚠ I nomi effettivi dei campi devono essere coerenti con quelli usati nel notebook e con le colonne attese dalla pipeline

⚠ I valori inseriti devono essere convertiti nei tipi corretti prima della previsione

Gestione del modello `.joblib`

Il file `.joblib` deve essere importato all'interno dell'app Flask.

Poiché il modello esportato contiene una **pipeline completa**, l'applicazione non deve:

- applicare manualmente scaler separati,
- applicare manualmente encoder separati,
- trasformare i dati con logiche diverse da quelle già incorporate nella pipeline.

L'applicazione deve semplicemente:

1. costruire i dati in formato tabellare,
2. passarli al metodo `predict()`,
3. interpretare il risultato.

Questo è coerente con il fatto che, nel progetto precedente, il modello è stato salvato come pipeline unica.

Comportamento richiesto

L'applicazione deve seguire questo flusso:

1. l'utente apre la pagina iniziale,
 2. compila il form con i dati richiesti,
 3. invia il form,
 4. Flask raccoglie i dati,
 5. i dati vengono organizzati in un `DataFrame`,
 6. il modello produce una previsione,
 7. in base all'esito:
 - se il mutuo risulta accettato → redirect a `success.html`
 - se il mutuo risulta rifiutato → redirect a `fail.html`
-

Template richiesti

1 `index.html`

Pagina principale dell'applicazione.

Deve mostrare:

- un titolo descrittivo del progetto
- un form con tutti i campi richiesti dal modello
- un pulsante di invio

Comportamento:

- alla sottomissione del form, i dati devono essere inviati alla route Flask che si occupa della previsione
-

2 `success.html`

Pagina mostrata quando la previsione del modello indica esito positivo.

Deve contenere:

- un messaggio chiaro di approvazione della richiesta
 - un link o pulsante per tornare alla pagina iniziale
-

3 `fail.html`

Pagina mostrata quando la previsione del modello indica esito negativo.

Deve contenere:

- un messaggio chiaro di rifiuto della richiesta
 - un link o pulsante per tornare alla pagina iniziale
-

◆ Route Flask richieste

Almeno le seguenti:

/

GET

→ mostra la pagina iniziale con il form (`index.html`)

/predict

POST

→ riceve i dati del form
→ costruisce la struttura dati attesa dal modello
→ carica o utilizza il modello `.joblib`
→ esegue la previsione
→ reindirizza verso:

- `success.html` in caso di esito positivo
- `fail.html` in caso di esito negativo

/success

GET

→ mostra la pagina `success.html`

/fail

GET

→ mostra la pagina `fail.html`

Gestione delle eccezioni (obbligatoria)

Devono essere gestiti almeno i seguenti casi:

- file `.joblib` non trovato
- errore nel caricamento del modello
- dati mancanti nel form

- dati in formato errato
- errore durante la previsione

L'app non deve crashare.

In caso di errore, è sufficiente:

- mostrare un messaggio chiaro,
- oppure reindirizzare l'utente alla pagina iniziale.

Questo requisito riprende il principio già presente nell'altra traccia: l'applicazione non deve interrompersi in presenza di errori gestibili.



Consegna richiesta

- progetto Flask funzionante
- file `.py` principali
- cartella `templates/` con i file HTML richiesti
- file `modello_mutuo.joblib`
- eventuali file di supporto necessari all'avvio del progetto